

download_data.py

Gold Price Historical Data Downloader

■ Overview

This script downloads historical Gold Futures price data from Yahoo Finance using the **yfinance** library. It saves the data as a CSV file so subsequent runs skip the download and load from disk instead — saving time and bandwidth.

■ Dependencies

- **yfinance** — fetches historical market data from Yahoo Finance
- **pandas** — handles data as a DataFrame and writes to CSV
- **os** — checks whether the CSV file already exists

■ Key Settings

DATA_FILE	gold_price_data.csv — name of the saved output file
Ticker	GC=F — Yahoo Finance symbol for Gold Futures
Date range	2015-01-01 → 2023-12-31 (9 years of daily data)

■■ How it Works — Step by Step

Step 1 — Check cache

If gold_price_data.csv already exists on disk, the function loads and returns it immediately without downloading.

Step 2 — Download

Calls `yf.download('GC=F', start='2015-01-01', end='2023-12-31')` to fetch 9 years of daily OHLCV data.

Step 3 — Validate

If the download returns an empty DataFrame (network error, bad ticker etc.) the function prints an error and returns None.

Step 4 — Flatten columns

Newer versions of yfinance return MultiIndex columns (Price, Ticker). The code flattens these by dropping the Ticker level.

Step 5 — Save & return

Saves the clean DataFrame to CSV and returns it to the caller.

■ Full Source Code

```
import yfinance as yf
import pandas as pd
import os

DATA_FILE = "gold_price_data.csv"

def download_data():
    if os.path.exists(DATA_FILE):
        print(f"{DATA_FILE} already exists. Skipping download.")
        return pd.read_csv(DATA_FILE, index_col='Date', parse_dates=True)

    print("Downloading historical gold prices...")
    ticker = "GC=F"
    gold_data = yf.download(ticker, start="2015-01-01", end="2023-12-31")

    if gold_data.empty:
        print("Failed to download data.")
        return None

    # Newer yfinance returns MultiIndex columns – flatten them
    if isinstance(gold_data.columns, pd.MultiIndex):
        gold_data.columns = gold_data.columns.droplevel('Ticker')

    gold_data.to_csv(DATA_FILE)
    print(f"Data saved to {DATA_FILE}")
    return gold_data

if __name__ == "__main__":
    df = download_data()
    if df is not None:
        print(df.head())
```

■■ How to Run

Run from the terminal inside the project directory:

```
python yuwa/download_data.py
```

On success it prints the first 5 rows of the gold price data and saves **gold_price_data.csv**.